

Multimodal Representation Learning for Vision-Language-Action Systems

Prepared on April 2, 2026

Abstract

This document packages a complete technical plan for a research program that starts from a 7B aligned language backbone, extends it into a grounded vision-language model (VLM), and then turns it into a vision-language-action (VLA) policy model for language-conditioned manipulation. The central research question is whether alignment choices made at the language-backbone stage materially affect later multimodal grounding quality and downstream action performance. The plan is organized as a sequence of large stages and week-sized substeps, with each stage producing a concrete model, a bounded dataset recipe, explicit evaluation metrics, and a repeatable experiment loop. The primary benchmark path remains CALVIN-centered for the core research comparison, while an integrated Isaac Lab validation platform provides a complementary engineering path for Franka arm manipulation using the same 7D action contract, image-plus-proprio observations, episode-safe datasets, evaluation metrics, and rollout GIFs. The purpose of the Isaac branch is to verify the embodied-action interface, rollout infrastructure, and continuous-control evaluation loop in a concrete robotics setting without replacing the main CALVIN benchmark path. The document also includes a data-construction protocol, metric definitions, and a practical compute plan centered on NVIDIA A40 rentals on Vast.ai under a monthly budget cap.

Stage overview

Stage	Primary model output	Core capability	Main evaluation
A	Aligned Language Backbone	Structured scene summaries, grounded reasoning, abstention, and action-plan generation	Format validity, tuple F1, plan validity, goal-reach accuracy
B	Grounded VLM	Image-conditioned QA, scene understanding, visual grounding, grounded multimodal summaries	VQAv2, GQA, RefCOCO(g), hallucination rate
C	Action-Aware VLM	Observation plus instruction to next low-level action	Action MAE/MSE, gripper accuracy, short-horizon consistency
D	CALVIN-centered VLA Policy	Language-conditioned multi-step manipulation policy	CALVIN average successful subtasks, Success@k, offline action error
E	Isaac Lab Validation Platform	Franka IK-relative continuous-control validation and rollout/evaluation infrastructure	Return, success rate, episode length, action jerk, dataset/GIF integrity

1. Project goal and research framing

- **Title.** Multimodal Representation Learning and Alignment for Vision-Language-Action Systems.
- **Main question.** Which alignment method produces a 7B language backbone that transfers best to grounded VLM behavior and then to VLA manipulation performance across both the core CALVIN benchmark path and the complementary Isaac Lab validation path?

- **Working hypothesis.** Alignment methods that improve schema compliance, uncertainty handling, and planning consistency at the text stage will yield better multimodal grounding and more stable action prediction downstream.
- **Deployment target.** A research stack that can ingest language instructions and visual observations, produce grounded internal reasoning, and emit low-level robot end-effector actions.
- **Validation role of Isaac Lab.** CALVIN remains the main benchmark for the research question, while Isaac Lab provides a complementary engineering validation platform for image-plus-proprio continuous-control manipulation with the same 7D action contract. This

Action representation used in the VLA stages

The action target is a low-level end-effector control vector rather than only macro labels. The recommended representation is $[\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{gripper}]$; a reduced version $[\Delta x, \Delta y, \Delta z, \Delta \text{yaw}, \text{gripper}]$ may be used in early action-aware experiments. The Isaac Lab validation platform uses the full 7D contract on *Isaac-Lift-Cube-Franka-IK-Rel-v0* so that the CALVIN-centered VLA path and the robotics validation platform speak the same action language.

Example task: Pick up the red block and place it inside the open drawer. The model should output a sequence of controls that first move above the red block, descend, close the gripper, lift, translate toward the drawer, then lower and open the gripper to release the object.

2. Large-step execution plan

Large Step 0 — Documentation, framing, and baseline artifacts

Stage output

A coherent research foundation consisting of the project statement, system architecture, README, experiment plan, and evidence-retention conventions.

Primary outputs

Project statement PDF, system architecture figure, experiment plan PDF, repository README, and weekly logging conventions.

0.1 — Draft the project statement

Goal. Define the project title, work pattern, scope, expected outputs, and relation to the field of study in one stable statement that can be reused across later reporting artifacts.

Expected outputs. Stable project statement and reporting-ready work description.

Experimental procedure.

1. State the project title, worksite, work pattern, research scope, expected outputs, and documentation practice in one compact statement.
2. Connect the project directly to computer vision, multimodal learning, and vision-language-action systems so the scope remains technically coherent.
3. Keep the wording stable so later weekly logs, reports, and archival materials can refer back to the same project description.

4. Retain the final PDF version as a standing reporting artifact.

0.2 — Create the system architecture and repository summary

Goal. Freeze the high-level visual explanation of the research pipeline and describe the repository's role in tracking the project.

Expected outputs. Stable architecture figure and coherent repository summary.

Experimental procedure.

1. Draw the full staged pipeline from language alignment through grounded VLM, action-aware modeling, CALVIN-centered VLA evaluation, and the complementary Isaac validation branch.
2. Name the canonical schemas, action representation, and evidence-retention components so the figure matches the written plan.
3. Write a repository summary that explains the main roadmap, data/evaluation artifacts, and retained documentation.
4. Store the figure and README under stable filenames for reuse.

0.3 — Freeze the baseline experiment plan and evidence structure

Goal. Produce the blueprint, define large stages and week-sized substeps, and state what records should be retained as evidence.

Expected outputs. Experiment plan PDF and named evidence-retention conventions.

Experimental procedure.

1. Define the large-step roadmap, stage outputs, datasets, metrics, and practical compute assumptions.
2. Map week-sized small steps so progress can be tracked consistently over time.
3. List the records to retain, including plans, logs, code history, evaluation reports, and compute receipts.
4. Freeze the document under a stable project filename so future work references the same blueprint.

Large Step 1 — Build the aligned 7B language backbone

Stage output

A text-only backbone optimized for strict structured output, concise scene summarization, grounded reasoning with correct abstention when evidence is missing, and short executable action-plan generation.

Primary training datasets

A mixed SFT corpus: a public instruction-following base, GQA scene-graph conversions for scene summaries and object states, GQA-derived answerable and intentionally unanswerable samples for grounded reasoning and abstention, and a symbolic planning corpus generated from a CALVIN-style manipulation world model. For preference training, combine a public chosen/rejected set with project-specific preference pairs generated from rubric scoring.

Primary evaluation metrics

Format Validity Rate (FVR), Field Exact Match (FEM), tuple F1 for groundedness, Plan Validity Rate (PVR), Goal-Round Accuracy (GRA), Abstention Accuracy (AA), and pairwise win rate against the SFT baseline.

1.1 — Choose the backbone and freeze the output interface

Goal. Select one 7B base model that is stable under constrained-format prompting and define the exact output schemas used throughout the project.

Recommended inputs. Two or three 7B candidates; a fixed 100-prompt probe set; JSON schemas for scene_summary, object_state_list, reasoning_answer, and action_plan.

Expected outputs. Chosen base model; canonical prompt templates; canonical JSON schemas; probe report showing formatting stability and answer behavior.

Experimental procedure.

1. Run the same 100 prompts on each candidate model with deterministic decoding and record raw outputs.
2. Score each model for parseable JSON rate, verbosity control, ability to keep required fields, and ability to answer 'insufficient information' when the prompt omits needed facts.
3. Choose the model with the best overall stability rather than the highest subjective eloquence.
4. Freeze four canonical output schemas so all later SFT, DPO, PPO, GRPO, and FIPO comparisons use the same interface.

1.2 — Train the SFT baseline

Goal. Create the first planning-friendly checkpoint and teach the base model to emit the required schemas consistently.

Recommended inputs. Selected 7B backbone; mixed SFT corpus; validation split containing all four task families.

Expected outputs. SFT checkpoint; validation report; list of failure modes such as hallucinated objects, missing fields, or overly long plans.

Experimental procedure.

1. Train on the mixed corpus with equal attention to schema-following tasks and planning tasks, not only generic instruction-following.
2. Evaluate after each epoch on a fixed validation set and compute FVR, FEM, tuple F1, PVR, GRA, and AA.
3. Inspect concrete failure examples and label them as format failure, groundedness failure, abstention failure, or planning failure.
4. Freeze the best validation checkpoint as the common starting point for preference and RL-style alignment.

1.3 — Train DPO on project-specific preferences

Goal. Improve the SFT model using chosen/rejected pairs that directly reward groundedness, schema correctness, and plan executability.

Recommended inputs. SFT checkpoint; public chosen/rejected data; project-specific pairs generated from rubric scoring.

Expected outputs. DPO checkpoint; DPO-versus-SFT comparison table; retained pair dataset with scoring metadata.

Experimental procedure.

1. Generate multiple candidate answers per prompt from the SFT checkpoint using varied temperatures or seeds.
2. Score each answer with a rubric that combines format correctness, grounded tuple F1, plan validity, and abstention correctness.

3. Construct chosen/rejected pairs only when the score gap is large enough to make the preference meaningful.
4. Train DPO and compare against SFT on pairwise win rate, FVR, PVR, GRA, and AA.

1.4 — Train PPO, GRPO, and FIPO variants

Goal. Compare online or RL-style alignment methods under the same planning-oriented reward structure.

Recommended inputs. SFT checkpoint; fixed prompt pool; scalar reward that scores schema correctness, groundedness, abstention, and valid action plans.

Expected outputs. PPO checkpoint; GRPO checkpoint; FIPO checkpoint; method comparison report identifying the best language backbone for transfer.

Experimental procedure.

1. Use the same prompt pool and the same reward dimensions for all methods so differences are attributable to the optimization method, not the task mix.
2. Track reward trajectory, KL behavior, FVR, PVR, GRA, and win rate versus the DPO and SFT checkpoints.
3. Reject models that improve reward numerically but degrade JSON validity or abstention behavior.
4. Select the one or two best aligned language backbones to carry into the VLM stage.

Large Step 2 — Convert the aligned backbone into a grounded VLM

Stage output

A VLM that can answer questions about images, summarize scenes in a task-relevant way, and provide grounded reasoning that later supports action prediction.

Primary training datasets

Image-caption warm-up data such as COCO Captions, multimodal instruction tuning data in the LLaVA style, and evaluation on VQAv2, GQA, and RefCOCO or RefCOCOg.

Primary evaluation metrics

VQAv2 accuracy, GQA standard accuracy, GQA consistency, RefCOCO(g) Acc@0.5 via IoU, and hallucination rate computed against oracle object sets.

2.1 — Attach the vision encoder and multimodal connector

Goal. Build a stable multimodal forward pass from image features into the aligned language backbone.

Recommended inputs. Best one or two aligned language backbones; one chosen vision encoder; connector design such as projector, adapter, or Q-Former-like bridge.

Expected outputs. Frozen multimodal architecture; sanity-check notebook proving image tokens influence generation.

Experimental procedure.

1. Freeze the vision encoder initially and train only the connector so the first experiment isolates the multimodal interface.

2. Run a tiny overfit test on a handful of image-caption samples to verify that image-conditioned outputs differ from text-only outputs.
3. Check whether image token masking causes output degradation; if it does not, the model is ignoring vision and the connector must be revised.
4. Lock the architecture before larger warm-up and instruction tuning runs.

2.2 — Warm up on image-caption data

Goal. Teach the model the basic mapping from image features to language so it can produce grounded descriptions before full visual instruction tuning.

Recommended inputs. Frozen multimodal architecture; COCO Captions or a similar image-caption corpus.

Expected outputs. Warm-start VLM checkpoint with basic captioning and object-mention ability.

Experimental procedure.

1. Train on image-caption supervision until the model reliably changes its output when the image changes.
2. Measure whether major objects, colors, and simple relations are mentioned in the generated summaries.
3. Use the canonical scene_summary schema where possible so the text interface remains aligned with the Stage-1 backbone.
4. Freeze the best warm-up checkpoint as the start point for instruction tuning.

2.3 — Run multimodal instruction tuning

Goal. Turn the warm-start VLM into an instruction-following visual assistant that stays grounded under image input.

Recommended inputs. Warm-start VLM; LLaVA-style multimodal instruction data; same output schemas used in Stage 1 where relevant.

Expected outputs. Instruction-following VLM checkpoint; qualitative set of image-grounded scene summaries and QA outputs.

Experimental procedure.

1. Train on image-question-answer and image-dialogue pairs while preserving concise, schema-compliant output.
2. Add task-relevant prompts that ask for scene_summary, salient_objects, and relevant_state fields under image input.
3. Reject configurations that produce fluent but visually unsupported answers, even if they look subjectively better.
4. Carry forward the best checkpoint judged by VQAv2, GQA, grounding behavior, and hallucination rate.

2.4 — Evaluate grounding and visual QA

Goal. Quantify whether alignment decisions from Stage 1 improved or harmed multimodal grounding.

Recommended inputs. All candidate VLM checkpoints; VQAv2, GQA, RefCOCO or RefCOCOg validation sets.

Expected outputs. VLM benchmark report; error taxonomy separating perception, reasoning, hallucination, and format-drift errors.

Experimental procedure.

1. Run VQAv2 accuracy using the official multi-annotator scoring protocol.
2. Run GQA accuracy and consistency to test compositional visual reasoning.
3. Run RefCOCO or RefCOCOg Acc@0.5 by measuring whether the predicted box reaches IoU at least 0.5 with the ground-truth region.
4. Compute hallucination rate by extracting mentioned objects and measuring which mentioned objects are absent from the oracle object set.

Large Step 3 — Build an action-aware VLM

Stage output

A multimodal model that consumes image or state observations plus a language instruction and predicts the next low-level robot action.

Primary training datasets

CALVIN as the main language-conditioned manipulation benchmark, with BridgeData V2 as an optional generalization check.

Primary evaluation metrics

Action MAE or MSE over continuous controls, gripper accuracy, and short-horizon sequence consistency under fixed history length.

3.1 — Canonicalize the action space

Goal. Define exactly what the model must predict at each time step so the action head and loss are well defined.

Recommended inputs. CALVIN and optional BridgeData V2 trajectories; chosen action dimensionality.

Expected outputs. Canonical action schema; parsers; converted training targets.

Experimental procedure.

1. Map raw dataset actions into a unified low-level control vector such as $[\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{gripper}]$ or a reduced 5D variant.
2. Standardize units and clipping ranges across datasets so action magnitudes are comparable.
3. Define the observation-plus-language to action-target format for single-step and short-sequence prediction.
4. Validate the parser by reconstructing a sample of original trajectories and checking numerical consistency.

3.2 — Train single-step next-action prediction

Goal. Prove that the VLM backbone can emit the correct immediate control signal before tackling multi-step policies.

Recommended inputs. Best VLM checkpoint; single-step CALVIN samples; canonical action schema.

Expected outputs. Action-aware VLM checkpoint; next-action baseline metrics.

Experimental procedure.

1. Freeze most of the VLM backbone and train only the action head or lightweight adapters at first.

2. Measure action MAE or MSE for continuous dimensions and exact accuracy for the gripper command.
3. Compare variants with and without the Stage-1 alignment gains to see whether better language alignment helps action prediction.
4. Keep the simplest architecture that achieves stable loss and meaningful action outputs.

3.3 — Add short observation history

Goal. Improve action stability by giving the model a short temporal window instead of a single frame.

Recommended inputs. Action-aware VLM; history windows such as $K=2$, 4, or 8; previous action tokens if available.

Expected outputs. History-aware checkpoint; short-horizon sequence-consistency report.

Experimental procedure.

1. Concatenate recent observations and optionally recent actions to the model input using a fixed history window.
2. Train and compare $K=2$, 4, and 8 to determine which temporal window gives the best trade-off between stability and compute.
3. Measure sequence consistency by checking whether all actions in a short horizon remain within a fixed error tolerance.
4. Freeze the best history length for the final VLA stage.

3.4 — Check cross-dataset generalization

Goal. Test whether the action-aware representation is overly specialized to one data source.

Recommended inputs. Best CALVIN-trained action-aware VLM; BridgeData V2 subset or another second manipulation source.

Expected outputs. Generalization report showing single-dataset performance versus transfer behavior.

Experimental procedure.

1. Keep the VLM backbone fixed and tune only small action adapters on the second dataset first.
2. Measure the same action metrics used on CALVIN so the numbers remain comparable.
3. Separate failures due to control mismatch from failures due to perception or instruction understanding.
4. Use the findings to decide whether a single-dataset final policy is sufficient or whether a mixed-data VLA is needed.

Large Step 4 — Train the CALVIN-centered VLA policy and run the transfer ablation

Stage output

A VLA policy model that maps observations plus a language instruction to action sequences and can be evaluated on language-conditioned manipulation tasks.

Primary training dataset

CALVIN trajectories and language instructions as the main policy benchmark, optionally supplemented with a second manipulation source for robustness checks.

Primary evaluation metrics

Offline action error, gripper accuracy, average number of successful subtasks on CALVIN sequences, and Success@k for long-horizon sequences.

4.1 — Fine-tune the VLA policy

Goal. Upgrade the action-aware VLM into a true policy model that predicts actions repeatedly over a task sequence.

Recommended inputs. Best action-aware VLM; CALVIN trajectory data; several freeze or unfreeze strategies.

Expected outputs. VLA policy checkpoints trained under frozen, partially unfrozen, and more adaptive settings.

Experimental procedure.

1. Start with the best action-aware VLM and train the policy head on full language-conditioned trajectories.
2. Compare a fully frozen backbone, partial unfreezing of upper layers, and a more adaptive configuration with lightweight adapters.
3. Track action losses and qualitative rollouts at regular intervals so the model does not drift silently.
4. Retain multiple checkpoints for the downstream transfer ablation.

4.2 — Run offline policy evaluation

Goal. Measure whether the policy predicts correct control signals on held-out trajectories before running expensive task-level evaluation.

Recommended inputs. Held-out CALVIN trajectories; all VLA candidate checkpoints.

Expected outputs. Offline policy report with per-dimension action error and sequence-level summaries.

Experimental procedure.

1. Compute continuous action MAE or MSE over each control dimension and overall.
2. Compute gripper accuracy for the binary open-versus-close command.
3. Measure short-horizon sequence consistency to catch unstable policies that look reasonable one step at a time.
4. Break results down by task family such as drawer operations, block relocation, or slider control.

4.3 — Run task-level CALVIN evaluation

Goal. Measure whether the model actually completes language-conditioned manipulation subtasks and sequences.

Recommended inputs. Best VLA checkpoints; CALVIN evaluation protocol.

Expected outputs. Task-level report with average successful subtasks and Success@k.

Experimental procedure.

1. Evaluate on fixed long-horizon sequences of language-conditioned subtasks.
2. Report the average number of consecutive subtasks completed correctly and the proportion of sequences that reach at least k completed subtasks.
3. Record representative successes and failures with the paired instructions and action traces.

4. Label each failure as primarily perception, planning, or control to guide follow-up work.

4.4 — Run the main transfer ablation

Goal. Answer the project's core question by comparing downstream performance while holding the vision and action stack fixed and swapping only the alignment backbone.

Recommended inputs. SFT-, DPO-, PPO-, GRPO-, and FIPO-derived backbones carried through the same VLM and VLA pipeline.

Expected outputs. Primary research conclusion about which alignment method transfers best to grounded VLM behavior and VLA policy quality.

Experimental procedure.

1. Keep the vision encoder, connector, action head, and data recipe identical across all variants.
2. Swap only the Stage-1 alignment backbone and re-run the VLM and VLA evaluation stack.
3. Compare text metrics, VQAv2 and GQA numbers, grounding accuracy, hallucination rate, and CALVIN task metrics side by side.
4. Identify whether any method gives strong text alignment but harms grounding or action performance, because that is the key negative result to capture.

Large Step 5 — Add the Isaac Lab arm-manipulation validation platform

Stage output

A concrete engineering validation branch built in Isaac Lab / Isaac Sim for a Franka Panda arm lifting a cube with continuous 7D control, image-plus-proprio observations, and a reproducible rollout / dataset / evaluation / GIF loop.

Primary roadmap files

plan_isaac_arm_manipulation.md acts as the primary roadmap for the full Isaac validation plan.

subplan_isaac_arm_manipulation.md acts as the finer-grained PR-level slice for the current implementation phase.

Why this validation branch is included

The CALVIN-centered path answers the main transfer question under a standardized benchmark, while the Isaac branch verifies that the same action semantics, observation interface, rollout storage, and evaluation loop remain operational in a real robotics runtime. This provides implementation confidence, surfaces infrastructure failures earlier, and creates reusable continuous-control evaluation artifacts without replacing the benchmark-centered core study.

Primary validation task

Isaac-Lift-Cube-Franka-IK-Rel-v0, with the action contract [dx, dy, dz, droll, dpitch, dyaw, gripper].

Primary evaluation metrics

Mean return, success rate, episode length, steps-to-threshold, mean action jerk, episode-safe dataset integrity, and rollout GIF artifacts.

5.1 — Create the validation repository scaffold (PR0)

Goal. Establish a stable repository structure, config defaults, reproducibility utilities, output directories, and smoke-test conventions for the Isaac branch.

Recommended inputs. Package directories for env, agents, train, dataset, configs, eval, and scripts; seed utilities; output helpers.

Expected outputs. Executable scaffold with deterministic utilities and tests.

Experimental procedure.

1. Add package directories and output folders for logs, checkpoints, data, GIFs, and plots.
2. Record dependency notes and environment assumptions for Isaac Sim, Isaac Lab, and PyTorch.
3. Add seed utilities and config defaults so later PRs share one stable foundation.
4. Verify imports, output directory creation, and deterministic seeds through scaffold tests.

5.2 — Define the task and action contract (PR1)

Goal. Freeze the exact Isaac Lab task and 7D action interface used by the validation platform.

Recommended inputs. Isaac-Lift-Cube-Franka-IK-Rel-v0; action_dim = 7; action names dx, dy, dz, droll, dpitch, dyaw, gripper.

Expected outputs. A stable task config with action clipping, gripper conversion, and camera requirements documented.

Experimental procedure.

1. Define action_dim = 7 and the exact action name order.
2. Clip all incoming actions to [-1, 1] and document the gripper open/close rule.
3. Require readable runtime errors when camera observations are requested without camera support.
4. Validate env id, action shape, clipping, and gripper semantics in task-contract tests.

5.3 — Build the formal Isaac Lab observation wrapper (PR2)

Goal. Convert Isaac Lab native observations into one stable project contract: {image, proprio}.

Recommended inputs. Configured Isaac Lab environment; explicit proprio feature order; batch-safe reset and step behavior.

Expected outputs. A formal adapter with image shape (num_envs, 3, 84, 84) and config-driven proprio vectors.

Experimental procedure.

1. Return image as uint8 RGB and proprio as float32 with explicit feature ordering.
2. Compute arm relative joint features, actual gripper finger state, end-effector position, cube position, target position, and derived relative vectors such as ee_to_cube and cube_to_target.
3. Support reset, step, close, done, truncated, and batch-safe behavior for num_envs = 1 and vectorized runs.
4. Verify shapes, dtypes, feature ordering, derived features, and readable camera/runtime errors in wrapper tests.

5.4 — Implement the shared image-propriio backbone (PR3)

Goal. Add the common encoder used by continuous-control RL agents and later diffusion models.

Recommended inputs. Image observations, proprio observations, configurable proprio_dim, fusion feature dimension.

Expected outputs. Reusable CNN + proprio MLP + fusion MLP backbone.

Experimental procedure.

1. Normalize images from uint8 to float inputs.
2. Encode image and proprio branches separately and fuse them into a shared feature representation.
3. Keep proprio_dim config-driven so the wrapper and models remain synchronized.
4. Verify output shapes and gradient flow through both branches.

5.5 — Train continuous-control RL baselines (PR4-PR7)

Goal. Build PPO, pure GRPO, SAC, and TD3 baselines on the same observation and action contract.

Recommended inputs. Shared backbone, Isaac wrapper, 7D continuous actions, and baseline-specific training loops.

Expected outputs. Comparable RL checkpoints and training curves, with SAC as the main expert-oracle candidate.

Experimental procedure.

1. Implement PPO with Gaussian tanh-squashed actions, value critic, GAE, and clipped objective.
2. Implement pure GRPO without a critic or value head, using group-relative returns.
3. Implement SAC with twin critics, replay buffer, entropy tuning, and deterministic oracle mode.
4. Implement TD3 with deterministic actor, target policy smoothing, delayed actor updates, and replay-buffer training.

5.6 — Collect episode-safe rollout and demonstration datasets (PR8)

Goal. Store rollouts by episode so future observation histories and action chunks never cross episode boundaries.

Recommended inputs. Heuristic, replay, or SAC-expert rollouts; HDF5 or zarr-style storage; metadata.

Expected outputs. Episode-safe dataset files containing images, proprios, actions, rewards, done/truncated flags, and metadata.

Experimental procedure.

1. Save images, proprios, actions, rewards, dones, truncateds, episode ids, and seed metadata.
2. Guarantee that future action-chunk sampling respects episode boundaries.
3. Validate file creation, key presence, shapes, metadata, and chunk-sampling safety.
4. Use the same schema for lightweight demo rollouts and later SAC demonstration collection.

5.7 — Train diffusion-policy imitation variants (PR9-PR10)

Goal. Distill expert behavior into smoother action chunks through behavior cloning and DAgger.

Recommended inputs. Episode-safe demonstration datasets, shared backbone conditioning, temporal U-Net diffusion policy, and SAC oracle labels.

Expected outputs. BC-Diffusion and DAgger-Diffusion checkpoints with action-chunk inference.

Experimental procedure.

1. Train a temporal diffusion policy on expert observation histories and future action chunks.
2. Use DDPM-style noise prediction loss and DDIM inference for chunked action generation.
3. Run DAgger by rolling out the student, querying the oracle on student-visited states, and aggregating new data.
4. Track demonstration count, oracle query count, and chunk-level rollout behavior.

5.8 — Evaluate, visualize, and package the Isaac validation branch (PR11-PR12 and one-command demo)

Goal. Provide one consistent evaluator and visual-output pipeline for random, heuristic, replay, RL, and diffusion agents.

Recommended inputs. All trained or scripted agents; evaluation loop; GIF recorder; comparison plots; one-command demo script.

Expected outputs. Metric JSON files, evaluation reports, rollout GIFs, and a reproducible interview-ready demo command.

Experimental procedure.

1. Compute mean return, success rate, mean episode length, steps-to-threshold, and mean action jerk.
2. Record rollout GIFs and optional side-by-side comparison grids with consistent overlays.
3. Support random, heuristic, replay, RL, and diffusion policies through the same data/evaluation interface.
4. Expose a one-command script that creates a dataset, metrics JSON, and GIF from the Isaac backend.

Large Step 6 — Package the project into a reusable research memory artifact

6.1 — Build the unified dashboard

Goal. Create a single place where text, VLM, CALVIN VLA, and Isaac validation metrics can be compared across checkpoints and branches.

Recommended inputs. All benchmark outputs and logs produced earlier.

Expected outputs. Unified result tables and figures that can be reused in reports and future chats.

Experimental procedure.

1. Normalize metric names and checkpoint naming across all stages and branches.
2. Build one master table for Large Steps 1-5 results.
3. Store plots and tables in stable filenames so future documents and chats can refer back to them consistently.
4. Preserve raw CSV or JSON exports so the dashboard can be regenerated.

6.2 — Write the error taxonomy

Goal. Turn qualitative failure analysis into a stable diagnosis system that spans text, grounding, planning, control, and validation-infrastructure failures.

Recommended inputs. Failure examples from text, VLM, CALVIN VLA, and Isaac validation evaluation.

Expected outputs. Error taxonomy document with alignment drift, grounding failure, planning failure, action/control failure, and infrastructure/runtime failure classes.

Experimental procedure.

1. Review representative failures from each stage.
2. Assign one dominant class and, when useful, a secondary class.
3. Write one paragraph per class explaining signature symptoms and likely causes.
4. Use the taxonomy to propose the next experiment instead of relying on ad hoc intuition.

6.3 — Produce the archive package

Goal. Generate a compact but complete artifact bundle that can be re-read later to recover the project state quickly.

Recommended inputs. Project plan, benchmark tables, compute assumptions, retained references, README, and weekly progress sheet.

Expected outputs. Final PDF blueprint, updated project statement, updated architecture figure, README, artifact index, and evidence pointers.

Experimental procedure.

1. Export the updated experiment plan and project statement as searchable PDFs.
2. Store benchmark definitions, dataset construction recipes, compute assumptions, and validation-platform details in the same archive.
3. Add references to the primary Isaac roadmap and the PR-level subplan so future readers know which file holds which level of detail.
4. Keep all files under stable names alongside code, logs, and weekly summaries.

3. Data construction protocol

3.1 Public instruction base

Use a public instruction-following SFT corpus as the generic instruction backbone. The public base is not the final task distribution; it only teaches the model to obey prompts. Sampling should favor constraint-following, concise responses, and structured-answer tasks rather than generic open-ended chatting.

3.2 GQA scene-graph conversions

Use GQA scene graphs to produce two deterministic text-supervision families. First, convert each graph into a short natural-language scene description and a canonical scene_summary or object_state_list target. Second, keep the original question-answer structure but verbalize the graph and require the model to answer from that verbalized state. This creates text-only grounded reasoning examples that still trace back to explicit object, attribute, and relation annotations.

3.3 GQA-derived abstention samples

Create intentionally unanswerable examples by removing the minimal fact needed to answer a valid GQA question. When the fact is removed, the target becomes 'insufficient information' with a short explanation. This yields a controlled abstention set instead of vague hand-written examples.

3.4 Symbolic planning corpus for action plans

Create a small symbolic manipulation world with objects such as colored blocks, a drawer, a slider, a button, an LED, and a gripper state. Sample legal world states and task templates such as 'open the drawer', 'place the red block in the drawer', or 'push the slider left'. Use a rule-based planner to generate preconditions, an ordered plan, and the expected goal state. This yields deterministic action-plan targets that stay structurally consistent with the later VLA domain.

3.5 Preference-pair construction

For each planning-oriented prompt, generate multiple candidate answers from the current checkpoint. Score each answer using a rubric that combines format correctness, groundedness, plan validity, and abstention correctness. The top-scoring answer becomes the chosen sample and the lowest-scoring answer becomes the rejected sample if the score margin is large enough. This prevents preference data from collapsing into near-ties.

3.6 Canonical output schemas

Canonical schemas used across the project:

scene_summary:

```
{"scene_summary": "...",  
"salient_objects": [...],  
"relevant_state": [...]}  
}
```

object_state_list:

```
{"objects": [{ "name": "...", "attributes": [...], "state": [...] }]}
```

reasoning_answer:

```
{"answer": "...",  
"evidence": [...],  
"confidence": "high | medium | low"  
}
```

action_plan:

```
{"feasible": true,  
"preconditions": [...],  
"plan": [...],  
"goal_state": [...]}  
}
```

3.7 Isaac Lab rollout and demonstration data contract

For the validation branch, store rollouts by episode with keys for images, proprio, actions, rewards, dones, truncateds, and metadata. The observation contract is {image, proprio} and the action contract remains 7D. This branch supports both lightweight demo rollouts and later SAC demonstration collection for diffusion-policy training.

4. Evaluation protocol and metric definitions

4.1 Text-backbone metrics

- **Format Validity Rate (FVR).** The fraction of outputs that are parseable and satisfy the canonical schema.
- **Field Exact Match (FEM).** For fields with deterministic answers such as feasibility or a binary state, compute exact match against ground truth.

- **Tuple F1.** Convert the prediction and target into sets of grounded tuples such as (object, attribute) or (object, relation, object), then compute precision, recall, and F1.
- **Plan Validity Rate (PVR).** For each generated action plan, count how many steps are valid under the symbolic world rules and divide by the total number of steps.
- **Goal-Reach Accuracy (GRA).** Execute the generated plan inside the symbolic simulator and record whether the final state matches the target goal state.
- **Abstention Accuracy (AA).** On intentionally answerable and intentionally unanswerable samples, measure whether the model abstains exactly when it should.
- **Pairwise win rate.** For DPO and RL-style methods, compare answers for the same prompt and record how often one model beats another under the rubric score.

4.2 VLM metrics

- **VQAv2 accuracy.** Use the official multi-annotator scoring protocol rather than plain exact match. A common shorthand is to cap agreement credit at one point once the predicted answer matches at least three annotators.
- **GQA standard accuracy and consistency.** Standard accuracy measures answer correctness; consistency checks whether the model answers related compositional questions coherently across a question set.
- **RefCOCO or RefCOCOg Acc@0.5.** Measure whether the predicted region reaches IoU of at least 0.5 with the ground-truth referred object.
- **Hallucination rate.** Extract object mentions from the response and measure the fraction that are absent from the oracle object set for that image or scene graph.

4.3 CALVIN action and VLA metrics

- **Action MAE or MSE.** Compute the error between predicted and ground-truth continuous control vectors over each action dimension and overall.
- **Gripper accuracy.** Compute exact-match accuracy on the gripper open-versus-close command.
- **Short-horizon sequence consistency.** Over a short time window, count whether every action remains within a fixed tolerance of ground truth.
- **CALVIN average successful subtasks.** Over long-horizon evaluation sequences, record how many consecutive subtasks are completed successfully on average.
- **Success@k.** Report the fraction of long-horizon sequences in which at least k subtasks are completed successfully.

4.4 Isaac Lab validation metrics

- **Mean return.** Compute return over evaluation episodes.
- **Success rate.** Measure whether the cube reaches the target height or other task-defined success signal.
- **Mean episode length and optional steps-to-threshold.** Report sample-efficiency and termination behavior.
- **Mean action jerk.** Compute the average $\|a_t - a_{(t-1)}\|_2$ over rollout steps as a smoothness proxy.
- **Dataset integrity checks.** Verify episode-safe storage, metadata completeness, and chunk-sampling safety.

- **Visual artifact checks.** Verify the existence and readability of rollout GIFs and, when applicable, side-by-side comparisons.

5. Compute plan: NVIDIA A40 on Vast.ai

Recommended hardware choice

Use an NVIDIA A40 on Vast.ai as the default rented GPU for this project. The A40 offers 48 GB of GDDR6 ECC memory, which is materially more comfortable than the RTX 3090's 24 GB for VLM and VLA experiments that combine a 7B language backbone with a vision encoder, temporal context, and action heads.

Why A40 instead of RTX 3090

A 24 GB RTX 3090 is sufficient for text-only SFT or DPO with LoRA or QLoRA and can handle lightweight VLM prototypes, but it becomes restrictive once image features, history windows, or action-policy heads are added. The A40 provides a more practical single-GPU ceiling for the later stages while still fitting a constrained research budget.

GPU	Useful role in this project	Practical fit
RTX 3090 (24 GB)	Text SFT, DPO, some PPO or GRPO, connector debugging, lightweight VLM prototypes	Usable early, tight later
A40 (48 GB)	Full-project default: text alignment, grounded VLM, action-aware VLM, and VLA fine-tuning with more comfortable batch and context budgets	Recommended default

Current price assumption

As checked on April 2, 2026, Vast.ai's public pricing page lists the A40 at about \$0.29 per hour. Under that base rate, a strict \$200 monthly compute cap corresponds to a theoretical maximum of about 689.7 A40 hours if no extra fees are incurred. In practice, reserve budget for storage, transfers, taxes, and occasional price fluctuations.

Monthly compute mode	A40 hours	Estimated GPU spend	Why use it
Lean	250 h	\$72.50	Enough for text alignment, dataset generation, and periodic VLM checks while leaving a large budget buffer.
Balanced	400 h	\$116.00	Comfortable default for ongoing SFT, DPO, VLM warm-up, and regular evaluation.
Aggressive	500 h	\$145.00	Good target when the project reaches sustained VLM and action-aware training.
Near-cap	600 h	\$174.00	Leaves only a small buffer; use only when long VLA runs dominate the month.

Recommended budget posture

Use the A40 as the default rented GPU and plan around the **Balanced** or **Aggressive** tiers. That keeps the project within the monthly cap while preserving meaningful room for storage and occasional market-price changes. The RTX 3090 remains useful for pure text experiments, but the A40 is the safer single-GPU default for the entire roadmap. The Isaac Lab validation branch follows the same overall budget posture while adding extra engineering considerations for headless runtime, camera support, and environment setup.

6. Unified weekly checkbox roadmap and current progress

Each checkbox corresponds to one week-sized small step. The core multimodal roadmap keeps the original large-step / small-step structure. The Isaac Lab validation branch is integrated as Large Step 5 and is finer-grained because the detailed subplan already exists as PR-level implementation units. Completed items are marked below.

Large Step 0 - Documentation and baseline artifacts

- ☒ Week 1 - 0.1 Draft the project statement
- ☒ Week 2 - 0.2 Create the system architecture and repository summary
- ☒ Week 3 - 0.3 Freeze the baseline experiment plan and evidence structure

Large Step 1 - Aligned 7B backbone

- ☐ Week 4 - 1.1 Choose the backbone and freeze the output interface
- ☐ Week 5 - 1.2 Train the SFT baseline
- ☐ Week 6 - 1.3 Train DPO on project-specific preferences
- ☐ Week 7 - 1.4 Train PPO, GRPO, and FIPO variants

Large Step 2 - Grounded VLM

- ☐ Week 8 - 2.1 Attach the vision encoder and multimodal connector
- ☐ Week 9 - 2.2 Warm up on image-caption data
- ☐ Week 10 - 2.3 Run multimodal instruction tuning
- ☐ Week 11 - 2.4 Evaluate grounding and visual QA

Large Step 3 - Action-aware VLM

- ☐ Week 12 - 3.1 Canonicalize the action space
- ☐ Week 13 - 3.2 Train single-step next-action prediction
- ☐ Week 14 - 3.3 Add short observation history
- ☐ Week 15 - 3.4 Check cross-dataset generalization

Large Step 4 - CALVIN-centered VLA policy

- ☐ Week 16 - 4.1 Fine-tune the VLA policy
- ☐ Week 17 - 4.2 Run offline policy evaluation
- ☐ Week 18 - 4.3 Run task-level CALVIN evaluation
- ☐ Week 19 - 4.4 Run the main transfer ablation

Large Step 5 - Isaac Lab validation platform

- ☒ Week 20 - 5.1 Create the validation repository scaffold (PR0)
- ☒ Week 21 - 5.2 Define the task and action contract (PR1)
- ☒ Week 22 - 5.3 Build the formal Isaac Lab observation wrapper (PR2)
- ☒ Week 23 - 5.4 Implement the shared image-proprio backbone (PR3)
- ☐ Week 24 - 5.5 Train PPO baseline (PR4, deferred)
- ☐ Week 25 - 5.5 Train pure GRPO baseline (PR5, deferred)
- ☒ Week 26 - 5.5 Implement SAC training infrastructure (PR6)
- ☒ Week 27 - 5.5 Implement TD3 training infrastructure (PR7)
- ☒ Week 28 - 5.6 Collect episode-safe rollout dataset (PR8-lite)
- ☐ Week 29 - 5.6 Collect full SAC demonstration dataset (PR8-full, pending)
- ☐ Week 30 - 5.7 Train diffusion-policy behavior cloning (PR9, pending)
- ☐ Week 31 - 5.7 Run DAgger fine-tuning (PR10, pending)
- ☒ Week 31 - 5.8 Add evaluation metrics, GIF/MP4 outputs, and checkpoint visual tools (PR11-PR12)

Status note - current SAC target-distance run is awaiting final results.

Large Step 6 - Research memory artifact

- ☐ Week 32 - 6.1 Build the unified dashboard
- ☐ Week 33 - 6.2 Write the error taxonomy
- ☐ Week 34 - 6.3 Produce the archive package

7. Archival notes for future reloading

This PDF is intended to function as a compact memory artifact for the full project. It preserves the multimodal alignment roadmap, integrates the CALVIN-centered VLA path with the Isaac Lab validation platform, and records the current progress state in one place. When re-uploaded later, it should allow the project context to be reconstructed without relying on prior chat memory. The primary Isaac roadmap file is *plan_isaac_arm_manipulation.md*, while *subplan_isaac_arm_manipulation.md* stores the finer PR-level execution detail for the current implementation slice.

External references checked on April 2, 2026

- [S1] Vast.ai public GPU pricing page: A40 listed at about \$0.29 per hour.
- [S2] NVIDIA A40 product page: 48 GB GDDR6 with ECC.
- [S3] NVIDIA GeForce RTX 3090 specifications page: 24 GB memory.
- [S4] Visual Question Answering evaluation page: official multi-annotator VQAv2 accuracy protocol.
- [S5] GQA pages: scene graphs, standard accuracy, and consistency definitions.
- [S6] CALVIN repository and benchmark description: language-conditioned long-horizon manipulation benchmark.
- [S7] RefCOCO-style grounding evaluation uses IoU-based accuracy at a threshold such as 0.5.

Prepared as a technical planning and archival document; not legal advice.